10. Colisiones (OnTriggerEnter2D, OnCollisionEnter2D y lógica de impactos)

Unity ofrece dos sistemas principales para detectar interacciones físicas: **Colliders con trigger** y **Colliders con colisión física**.

10.1. Tipos de colisión en 2D

OnTriggerEnter2D()

Se activa cuando dos colliders se tocan **y al menos uno tiene isTrigger = true**.

Uso ideal para:

- Balas
- Zonas de daño
- Detectar recogidas
- Enemigos que desaparecen al chocar

OnCollisionEnter2D()

Se activa cuando dos colliders se tocan **y ninguno es trigger**.

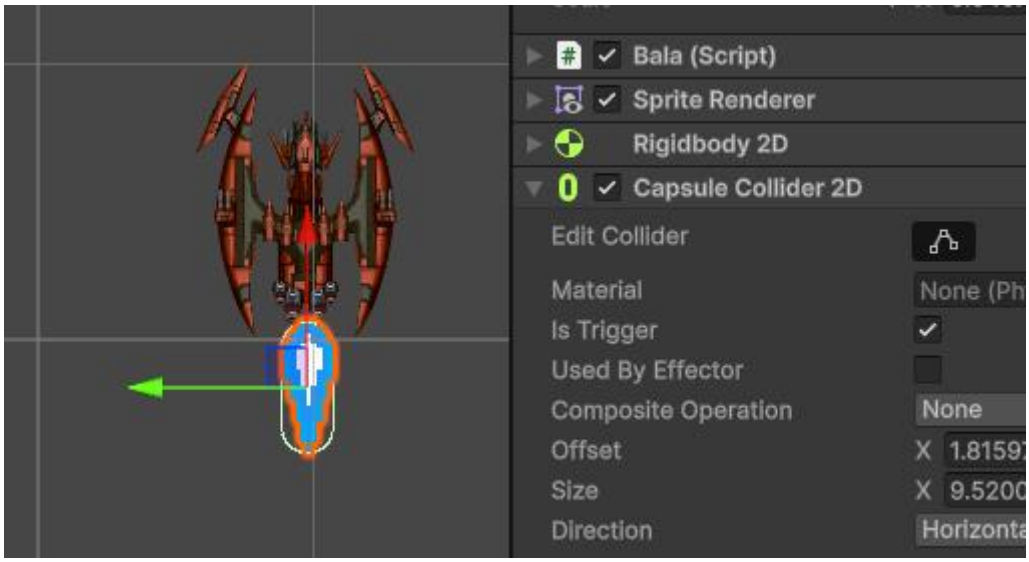
Uso típico para:

- Jugador pisando el suelo
- Objetos sólidos que chocan físicamente

10.2. Configuración necesaria para que funcione una colisión

Para que Unity detecte una colisión 2D:

1. Ambos objetos deben tener **Collider2D**
2. Al menos uno debe tener **Rigidbody2D**
3. Si se quiere trigger: activar **isTrigger**
4. Si se quiere colisión física: dejar **isTrigger desactivado**



10.3. Colisiones en los scripts del proyecto

10.3.1. Colisiones de la bala del jugador

En el script **BalaJugador**, la bala usa trigger:

```
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.CompareTag("Enemigo"))
    {
        Destroy(collision.gameObject);
        Destroy(gameObject);
    }
}
```

Acciones realizadas:

- Detecta al enemigo
- Destruye al enemigo
- Se autodestruye

10.3.2. Colisión de bala enemiga con el jugador

En el script **BalaEnemigo**:

```
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.CompareTag("Jugador"))
```

```
    {
        collision.GetComponent<Jugador>().RecibirDaño(daño);
        Destroy(gameObject);
    }
}
```

10.3.3. Colisión del jugador con enemigos

En el script **Enemigo**:

```
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.CompareTag("Jugador"))
    {
        collision.GetComponent<Jugador>().RecibirDaño(1);
    }
}
```

10.3.4. Rocas generadas por el GameManager

Las rocas detectan colisión con balas y jugador usando triggers configurados en sus colliders.

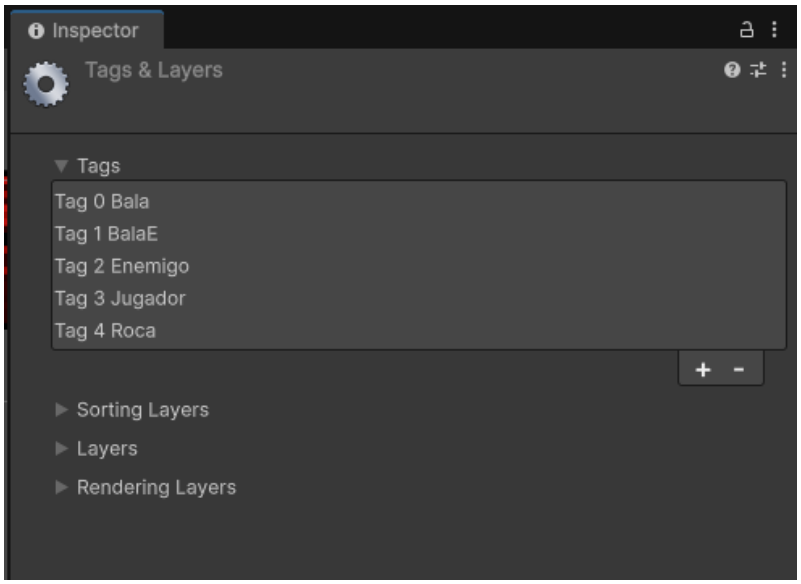
10.4. Uso de etiquetas (tags) en colisiones

Para identificar qué tipo de objeto se ha tocado se usa:

collision.CompareTag("Jugador")

Etiquetas empleadas en este proyecto:

- "Jugador"
- "Enemigo"
- "BalaJugador"
- "BalaEnemigo"
- "Roca"



10.5. Orden de ejecución y problemas comunes

Problema 1: "OnTriggerEnter2D no funciona"

Causas posibles:

- El objeto no tiene Rigidbody2D
- El collider no está en modo trigger
- El objeto está desactivado
- No coincide el tag esperado

Problema 2: "La bala atraviesa objetos"

Solución:

- Activar **Continuous** en el Rigidbody2D de la bala
- Reducir la velocidad
- Asegurar trigger o collider correctamente

Problema 3: "Las colisiones físicas empujan demasiado"

Solución:

- Usar triggers en lugar de colisiones físicas
- Ajustar masa y drag
- Usar materiales físicos con poco bounciness

10.6. Buenas prácticas

- Usar triggers siempre que se pueda, son más ligeros
- Mantener tags organizadas
- Evitar colisiones innecesarias desactivando colliders cuando no se usen
- Destruir objetos después de colisionar para evitar dobles impactos